

OPEN-SOURCE RESEARCH REPORT

OpenCode Config, Skills, Agent Tools Research

Comprehensive analysis of 30+ open-source projects and architectural patterns for improving configuration, skills, agent customization, and plugin capabilities in the Zakarialabib/opencode ecosystem.

Prepared for: Zakarialabib/opencode

Focus: Android Tools, Agent Config, Skill Abstraction

Date: May 2026

Table of Contents

1. Executive Summary	3
2. OpenCode Current State Analysis	3
2.1 Architecture Overview	3
2.2 Current Tool/Agent/Config System	4
2.3 Critical Gaps	4
3. AI Agent Config Frameworks & Registries	5
3.1 Skill/Tool Registries	5
VoltAgent / awesome-agent-skills	5
ToolRegistry (Oaklight)	6
AgentRegistry (Solo.io)	6
MCP Official Registry	6
3.2 Plugin Systems for AI Coding Assistants	7
Goose (Block / AAIF / Linux Foundation)	7
Continue (continuedev/continue)	7
Cline	7
3.3 Agent Configuration and Persona Frameworks	8
CrewAI	8
Caliber (caliber-ai-org)	8
OpenHands (All-Hands-AI)	8
4. Skill/Provider Abstraction Patterns	9
4.1 Config Format Comparison	9
4.2 Provider Abstraction Best Practices	10
4.3 Tool/Skill Definition Hierarchy	10

4.4 Cross-Cutting Patterns Summary	11
5. Android-Aware AI Tooling Ecosystem	12
5.1 Official Google Android Tooling	12
Android CLI + Android Skills	12
Android Studio Gemini Agent Mode + MCP Support	13
5.2 Android MCP Servers	13
5.3 Android Skill Definitions for AI Agents	14
amit-nayar/android-adb-skill	14
JetBrains Koog	14
6. Integration Recommendations & Roadmap	15
6.1 Phase 1: Quick Wins (1-2 Weeks)	15
6.2 Phase 2: Structural Improvements (2-4 Weeks)	15
6.3 Phase 3: Ecosystem Integration (4-8 Weeks)	16
6.4 Recommended Android Integration Architecture	16
7. Complete Project Reference	17
7.1 Agent Config & Registry Projects	17
7.2 Android-Aware Tooling Projects	18
8. Key Takeaways	19

1. Executive Summary

This report presents a comprehensive research analysis of open-source projects and architectural patterns that can significantly improve the configuration, skills, agent, tools, and plugin capabilities of the **Zakarialabib/opencode** project. The research was motivated by three critical gaps identified in the current OpenCode architecture: the absence of Android-aware tool definitions, the lack of agent instruction customization mechanisms, and the missing skill/provider abstraction layer that would enable a plugin or skill registry pattern to be wired into the OpenCode server configuration.

The Zakarialabib/opencode project is a heavily extended fork of the original opencode-ai/opencode (now archived and continued as charmbracelet/crush). While the upstream project provided just 2 built-in agents and basic tool support, the Zakarialabib fork has evolved into a full multi-agent orchestration platform featuring 12+ specialized agents, 63+ skills, YAML-defined workflow pipelines, RAG context retrieval, 8+ MCP servers, and 10+ TypeScript plugins. Despite this impressive expansion, the project still lacks structured abstractions in three key areas that this report addresses with concrete, actionable recommendations drawn from the most successful open-source projects in the AI agent ecosystem.

Across 30+ open-source projects analyzed, several dominant architectural patterns emerge. The **SKILL.md format** (YAML frontmatter + Markdown body) is becoming the de facto standard for skill definitions, adopted by Claude Code, Codex, Gemini CLI, and Goose. The **MCP (Model Context Protocol)** is rapidly becoming the universal tool integration standard, with nearly every major agent framework now supporting it. **YAML configuration** is preferred over JSON or TOML for its readability and comment support. And **protocol-agnostic tool registries** are emerging as the solution to fragmentation between OpenAI function calling, MCP, and OpenAPI formats. This report provides detailed project profiles, architectural pattern analysis, and a prioritized integration roadmap specifically tailored for OpenCode.

2. OpenCode Current State Analysis

Before examining external projects, it is essential to establish a clear picture of the current OpenCode architecture, its strengths, and its gaps. This section provides a detailed technical assessment of the Zakarialabib/opencode fork as it exists today, focusing on the three areas identified as needing improvement: Android-aware tooling, agent instruction customization, and skill/provider abstraction.

2.1 Architecture Overview

OpenCode uses a client/server architecture with a JavaScript (Bun runtime) HTTP server built on Hono and a Go TUI (Bubble Tea) client. LLM integration leverages the Vercel AI SDK for provider-agnostic model access. The Zakarialabib fork introduces a "Swarm Architecture" where specialized agents collaborate under a lead-strategist orchestrator, organized into categories: Core (core-factory), Lead

(lead-strategist, lead-architect), Backend (backend-api, backend-laravel, backend-tauri), Frontend (frontend-ui-ux), QA (qa-guardian), DevOps (devops-engineer), and Docs (docs-curator). Additionally, there are agent definitions for Android/Kotlin, build, explore, plan, and scout workflows.

Component	Technology	Notes
Backend Server	JavaScript (Bun) + Hono	HTTP API server
TUI Client	Go + Bubble Tea	Terminal UI (charmbracelet)
LLM Integration	Vercel AI SDK	Provider-agnostic model access
Database	SQLite	Sessions, messages, metrics
RAG/Brain	nomic-embed + Rust sidecar	Local embeddings + HNSW vector search
MCP	Model Context Protocol	8+ pre-configured MCP servers
LSP	Language Server Protocol	Code intelligence integration
Plugin System	TypeScript plugins	10+ plugins including agent-router

Table 1. OpenCode Technology Stack

2.2 Current Tool/Agent/Config System

The primary configuration file is **opencode.json** (34KB), which uses JSON format and references a schema at opencode.ai/config.json. It defines the default model (inclusionai/ring-2.6-1t:free via OpenRouter), providers, default agent (core-factory), plugins array, MCP server declarations, and per-agent prompts and model selections. The upstream project provides built-in tools including glob, grep, ls, view/read, write, edit, patch, bash, fetch, diagnostics, agent, and sourcegraph. The fork adds 8+ MCP servers (context7, filesystem, memory, git, fetch, sqlite, sequential-thinking, language-server) and 63+ skills organized in directories with SKILL.md files using YAML frontmatter.

The skill system uses SKILL.md files with YAML frontmatter containing name, description, license, compatibility, and metadata fields. The 47 skill directories include development skills (laravel-feature-scaffold, coding-agent, stack-context), content skills (deep-research, web-reader, web-search), analysis skills (security-review, testing-strategy, database-design), automation skills (workflow-manager, self-improver, dynamic-workflow), media skills (pdf, ppt, xlsx, docx, TTS), and research skills (market-research-reports, aminer academic). Workflows are defined in YAML format (v2.0.0 schema) with phases, agent routing, MCP tool declarations, and parallel execution groups.

2.3 Critical Gaps

Despite the impressive scope of the fork, three structural gaps remain that limit the project's extensibility and its ability to support domain-specific workflows like Android development:

Gap	Current State	Impact	Priority
No Android-aware tools	android-kotlin.md is a 929-byte prompt definition only; no ADB, Gradle, or Android SDK integration	Cannot perform build, deploy, test, or debug cycles for Android projects	Critical
No agent customization framework	Agents are markdown files with prompts; no programmatic API for composing agents with custom state machines, tool sets, or guardrails	Cannot build domain-specific agent behaviors beyond prompt engineering	High
No skill/provider abstraction	Skills are markdown directories without a unified interface, versioning, dependency resolution, or testing framework	Cannot compose skills programmatically, enforce quality, or resolve conflicts	High
Config format fragmentation	JSON (main config), YAML frontmatter (skills), YAML (workflows), Markdown (agents), TypeScript (plugins)	No single source of truth; developers must context-switch across formats	Medium
No SDK for external integrations	While upstream has @opencode-ai/sdk, the fork lacks a clear API for external tools to register agents/skills	Third-party integrations require forking the codebase	Medium

Table 2. Critical Gaps in Current OpenCode Architecture

3. AI Agent Config Frameworks & Registries

This section surveys the most relevant open-source projects that provide frameworks, registries, and patterns for AI agent configuration, tool management, and skill registries. Each project is evaluated for architectural patterns that could be adopted or adapted for OpenCode.

3.1 Skill/Tool Registries

VoltAgent / awesome-agent-skills

VoltAgent maintains a curated collection of 1,000+ agent skills from official dev teams and the community, compatible with Claude Code, Codex, Gemini CLI, Cursor, and other AI coding agents. The project follows a pure "Awesome-list" pattern where each skill is a text file with structured instructions that teach AI agents new capabilities on demand. The sibling project,

awesome-openclaw-skills, catalogs 5,400+ skills from the OpenClaw Skills Registry. This approach demonstrates that community-driven skill catalogs can scale to thousands of entries with minimal governance overhead. For OpenCode, adopting a similar registry pattern for skills would enable third-party developers to contribute Android-specific, Kotlin-specific, and Gradle-specific skills without modifying the core codebase. The key architectural pattern here is the **Curated Registry**: a directory of reusable instruction files indexed by topic, framework, and compatibility tags.

GitHub: <https://github.com/VoltAgent/awesome-agent-skills>

ToolRegistry (Oaklight)

ToolRegistry is a protocol-agnostic tool management library for function-calling LLMs that simplifies tool registration, representation, execution, and lifecycle management. It is one of the most architecturally relevant projects for OpenCode because it solves the exact problem of tool fragmentation across multiple LLM function calling formats. The library provides a unified register/generate-schema/execute pipeline where tools are registered once with Python type annotations and automatically exported to OpenAI, Anthropic, Gemini, LangChain, and MCP formats. This achieves a 60-80% reduction in tool integration code. The three-package ecosystem (core registry, server library with OpenAPI + MCP support, and a ready-to-use tool collection) demonstrates a clean separation of concerns. For OpenCode, the **Protocol-Agnostic Registry + Provider Pattern** is the key takeaway: tools should be defined once and exposed via multiple protocols, rather than requiring separate definitions for each LLM provider format.

GitHub: <https://github.com/Oaklight/ToolRegistry> | **PyPI:** v0.9.1 | **arXiv:** 2507.10593

AgentRegistry (Solo.io)

AgentRegistry, backed by Solo.io, is an open-source platform to find, manage, and run MCP servers, AI agents, and skills from a single governed registry. It supports JWT validation, on-behalf-of authentication flows, and centralized observability and logging of agent-to-tool traffic. The project uses OCI-based packaging for agent distribution, making it container-native. While the enterprise/commercial focus may be heavier than what OpenCode needs, the **Governed Registry / App Store Pattern** is valuable: a single curated registry with authentication, validation, and observability for all AI artifacts. For OpenCode, a lightweight version of this pattern could provide skill provenance tracking, version validation, and usage analytics without the enterprise overhead.

GitHub: <https://github.com/agentregistry-dev/agentregistry> | **Backed by:** Solo.io

MCP Official Registry

The Model Context Protocol's official registry is the authoritative repository for publicly-available MCP servers. API v0.1 is in freeze (stable). It functions as an "app store" for MCP servers where clients can discover, install, and configure MCP servers. The official registry is critical for OpenCode because MCP is becoming the universal tool integration standard, and the registry provides the discovery and

distribution layer. OpenCode already supports MCP servers in its configuration, so wiring into the official registry for auto-discovery and one-click installation of MCP servers (especially Android-related ones like the Gradle MCP Server and ADB MCP servers discussed in Section 5) would be a high-impact, relatively low-effort improvement. The **Official Registry / App Store** pattern ensures that OpenCode users can discover and install tools without manually editing JSON configurations.

GitHub: <https://github.com/modelcontextprotocol/registry>

3.2 Plugin Systems for AI Coding Assistants

Goose (Block / AAIF / Linux Foundation)

Goose is an open-source, extensible AI coding agent with 70+ extensions via MCP, now part of the Agentic AI Foundation at the Linux Foundation. With 38k+ GitHub stars, it is one of the most popular and actively maintained agent frameworks. Goose uses YAML configuration (`~/.config/goose/config.yaml`) with a clean separation of concerns: extensions are MCP servers defined in the config with name, type (stdio), command, args, and environment variables; skills are markdown-based instruction files (SKILL.md pattern) with YAML frontmatter; recipes are YAML-based reusable agent configurations; and custom providers are stored as JSON files in `~/.config/goose/custom_providers/`. The key patterns for OpenCode are: (1) **YAML config for providers + extensions** with clean separation, (2) **SKILL.md pattern** for cross-tool compatible skill definitions, (3) **MCP as the extension protocol** for first-class tool discovery, and (4) **Custom providers as self-describing JSON files** that can be added without code changes.

GitHub: <https://github.com/aaif-goose/goose> | **Stars:** 38k+

Continue (continuedev/continue)

Continue is a leading open-source AI code assistant with 24k+ stars that migrated from JSON to YAML configuration. Its `config.yaml` demonstrates several best practices directly applicable to OpenCode. The `models` section uses name, provider, model, apiBase, roles (an array of capabilities like chat, edit, apply, summarize, autocomplete, embed, rerank), and a capabilities array (`tool_use`, `image_input`) for declarative feature detection with auto-detection fallback. Schema versioning (`schema: v1`) enables backward-compatible config evolution. MCP servers are declared in a dedicated `mcpServers` section with `command/args/env`. Prompt templates are defined per role (`chatOptions.baseSystemMessage`, `baseAgentSystemMessage`, `basePlanSystemMessage`). `RequestOptions` allow per-model HTTP configuration including timeout, SSL, proxy, headers, and `extraBodyProperties`. The most important patterns for OpenCode are: (1) **Model roles and capabilities arrays** for declarative feature detection, (2) **Schema versioning** in YAML for backward-compatible config evolution, and (3) **Prompt templates per agent role** rather than hardcoded system messages.

GitHub: <https://github.com/continuedev/continue> | **Stars:** 24k+

Cline

Cline is an autonomous coding agent available as an SDK, IDE extension (VS Code, JetBrains), CLI, and Kanban board with 61k+ GitHub stars and 5M+ installs. It features a built-in MCP Marketplace for discovering and installing MCP servers with one click, project-level .clinerules files for behavioral configuration, and a Plan/Act mode that separates reasoning from execution. The SDK enables custom tools and multi-agent orchestration. The **MCP Marketplace + SDK Pattern** is the key takeaway: a centralized discovery layer for tool servers combined with an SDK that makes the agent engine reusable across different interfaces. For OpenCode, building an MCP marketplace (even a simple community-curated list) would dramatically reduce the friction of adding new tools and servers.

GitHub: <https://github.com/cline/cline> | **Stars:** 61k+ | **License:** Apache 2.0

3.3 Agent Configuration and Persona Frameworks

CrewAI

CrewAI is a framework for orchestrating role-playing, autonomous AI agents where each agent is defined by a Role-Goal-Backstory persona definition. Agents have per-agent tool assignments, and crews (teams) are defined declaratively in YAML. The CrewAI Skills repository provides structured markdown instructions for teaching agents specific workflows, while the CrewAI Tools package provides 50+ integrations. The **Persona-first Agent Config + Tool Assignment** pattern is particularly relevant for OpenCode: instead of defining agents as raw markdown prompts, agents should have structured persona definitions (role, goal, backstory) with explicit tool assignments. This makes agents more composable and testable. For Android development, this would mean defining an Android agent with role="Android Developer", goal="Build, test, and deploy Android applications", and tools assigned from an Android toolkit (ADB, Gradle, Kotlin compiler).

GitHub: <https://github.com/crewaiinc/crewai> | **Stars:** 30k+

Caliber (caliber-ai-org)

Caliber is a CLI that fingerprints projects and generates/syncs AI agent configs (CLAUDE.md, skills, AGENTS.md, MCPs) across Claude Code, Cursor, and Codex. It features a community registry for AI agent configuration files, auto-generates agent skills, MCP configs, and instruction files based on project context, and continuously syncs them as the codebase evolves. The **Config Generation + Sync Registry** pattern is directly applicable to OpenCode: instead of requiring users to manually craft opencode.json, skills, and agent definitions, the system could auto-generate these based on project detection (e.g., detecting an Android project and automatically configuring ADB tools, Gradle MCP server, and Android-specific agent prompts). This would dramatically lower the barrier to entry for new users and ensure that configurations stay in sync with project changes.

GitHub: <https://github.com/caliber-ai-org/ai-setup>

OpenHands (All-Hands-AI)

OpenHands is a 54k+ star agent framework with a clean Action/Observation loop architecture. Each Action is a typed dataclass, and the AgentComputerInterface (ACI) provides a clean separation between tool interface and agent logic. The stateless agent core emits typed Actions, and an append-only EventLog captures conversation history. The SDK architecture enables composable, modular deployment from local to production. The key lesson for OpenCode is the **Action/Observation typed loop + ACI abstraction**: tools should communicate through typed action/observation pairs rather than raw string inputs/outputs. This makes tool composition safer and more predictable, especially when chaining tools (e.g., Gradle build action producing a build output observation that feeds into an ADB deploy action). The cautionary lesson is that OpenHands' dual config system (TOML + JSON) is being unified, confirming that config format fragmentation is a real problem worth solving early.

GitHub: <https://github.com/All-Hands-AI/OpenHands> | **Stars:** 54k+

4. Skill/Provider Abstraction Patterns

This section analyzes the architectural patterns used by leading projects to abstract skills, providers, and tools into composable, extensible systems. These patterns are the building blocks for solving OpenCode's third critical gap: the lack of a skill/provider abstraction layer.

4.1 Config Format Comparison

Across all projects surveyed, YAML has emerged as the dominant configuration format for AI agent systems, followed by the SKILL.md format (YAML frontmatter + Markdown body) for skill definitions. JSON is used primarily where schema validation is critical (MCP tool schemas), and TOML is being phased out (OpenHands is unifying from TOML+JSON to a single format). The following table summarizes the format choices across major projects:

Project	Main Config	Skills Format	Agent Format	Why
Goose	YAML	SKILL.md	YAML recipes	Human-readable, comments, extensible
Continue	YAML	Rules + prompts	Model roles	Migrated from JSON; schema versioning
OpenHands	TOML+JSON (unifying)	Python classes	Python dataclasses	Being unified to single format
CrewAI	YAML	Markdown skills	YAML crews	Declarative crew definitions
Docker cagent	YAML	YAML tools	YAML agents	Pure YAML, zero-code agents

Claude Code	JSON settings	SKILL.md	CLAUDE.md	SKILL.md = emerging standard
Cline	VS Code JSON	.clinerules	SDK	IDE-coupled settings
MCP	JSON	JSON Schema	Tool schemas	Schema validation critical

Table 3. Config Format Comparison Across Projects

Recommendation: OpenCode should adopt **YAML for main configuration** (replacing or supplementing the current JSON format) and **SKILL.md for skill definitions** (which it already partially uses). This aligns with the emerging industry standard and provides the best balance of readability, extensibility, and cross-tool compatibility.

4.2 Provider Abstraction Best Practices

The consensus from analyzing multiple LLM abstraction layers is clear: over-abstracting providers loses provider-specific features. The most successful projects follow a minimal interface approach: normalize only the request/response layer, use string-based provider identifiers ("openai", "anthropic", "ollama") mapping to adapter classes, support apiBase override for OpenAI-compatible endpoints, and declare capabilities declaratively (tool_use, image_input, streaming) rather than detecting them at runtime. The following principles should guide OpenCode's provider abstraction:

- **Normalize request/response only** - do not hide provider-specific features like Claude's extended thinking or OpenAI's structured outputs
- **String-based provider identifiers** - "openai", "anthropic", "ollama" mapping to adapter classes, not factory patterns
- **Declarative capabilities** - [tool_use, image_input, streaming] array per model, auto-detected with manual override
- **apiBase override** - support for OpenAI-compatible endpoints (LM Studio, vLLM, Ollama)
- **Per-model HTTP config** - timeout, SSL, proxy, headers, extraBodyProperties (from Continue)
- **Custom providers as self-describing files** - JSON/YAML files in a providers/ directory (from Goose)

4.3 Tool/Skill Definition Hierarchy

Across all projects analyzed, a consistent four-level hierarchy emerges for organizing agent capabilities. At the lowest level are atomic **Tools** (single actions like "run bash command" or "read file"). Tools are grouped into **Toolkits/Plugins** (related tools for a domain, like an "Android Toolkit" containing ADB, Gradle, and Kotlin compiler tools). Toolkits are referenced by **Skills** (behavioral instructions that combine tool references with context, examples, and workflow guidance). At the top level, **Agents**

combine skills, tools, and provider configuration into a complete persona with a defined role, goal, and backstory. This hierarchy provides a natural extension path: new tools can be added independently, toolkits can be composed from existing tools, skills can reference multiple toolkits, and agents can combine multiple skills. For OpenCode, this hierarchy maps cleanly to the existing architecture: the built-in tools (bash, edit, grep, etc.) are already at the Tool level; MCP servers provide Toolkits; SKILL.md files define Skills; and agent markdown files define Agents. The missing piece is the formalization of this hierarchy with explicit references and dependency resolution between levels.

Level	OpenCode Current	Recommended Enhancement
Tool (atomic action)	Built-in tools (bash, edit, grep)	Formal tool interface with JSON Schema input/output types
Toolkit (tool group)	MCP servers	Named toolkits with explicit tool membership; auto-register as MCP tools
Skill (behavior + tools)	SKILL.md directories	YAML frontmatter with toolkit_refs field; dependency resolution
Agent (skills + provider)	Agent markdown files	Structured YAML definition with role/goal/backstory + skill_refs + model config

Table 4. Tool/Skill Definition Hierarchy: Current vs. Recommended

4.4 Cross-Cutting Patterns Summary

The following table summarizes the key architectural patterns discovered across all projects and their applicability to OpenCode. Each pattern is rated by implementation complexity and expected impact:

Pattern	Source Projects	OpenCode Application	Complexity	Impact
SKILL.md as standard	Claude Code, Goose, CrewAI	Unify skill format across all agents	Low	High
YAML main config	Goose, Continue, CrewAI, cagent	Replace/supplement JSON config	Medium	High
MCP as extension protocol	Goose, Cline, OpenAI SDK	Formalize MCP as primary tool extension	Low	High
Protocol-agnostic tool registry	ToolRegistry (Oaklight)	Define tools once, export to multiple formats	High	High

Model roles + capabilities	Continue	Declarative feature detection per model	Medium	Medium
Schema versioning	Continue	Backward-compatible config evolution	Low	Medium
Agent Card / persona	A2A, CrewAI	Structured agent descriptions for discovery	Medium	High
Config auto-generation	Caliber	Auto-generate configs from project context	High	High
Action/Observation typed loop	OpenHands	Typed tool inputs/outputs for safer composition	High	Medium
Custom providers as files	Goose	Self-describing provider JSON/YAML files	Low	Medium

Table 5. Architectural Patterns: Applicability to OpenCode

5. Android-Aware AI Tooling Ecosystem

This section surveys the rapidly growing ecosystem of Android-specific AI agent tools, MCP servers, and frameworks. The Android AI tooling landscape has matured significantly in 2025-2026, with Google itself launching official agent-first tooling and multiple community MCP servers bridging ADB, Gradle, and the Android SDK to LLM workflows. These projects directly address OpenCode's first critical gap: the absence of Android-aware tool definitions.

5.1 Official Google Android Tooling

Android CLI + Android Skills

Google has officially entered the agent-first tooling space with the Android CLI and Android Skills repository. The Android CLI is a purpose-built command-line interface for agent-first Android development that produces structured JSON output that AI agents can parse. It wraps core Android development operations (build, test, lint, emulator control) into a machine-readable interface. The Android Skills repository provides modular, markdown-based (SKILL.md) instruction sets covering Compose, Gradle, testing, architecture, and other Android topics. These are the official best-practices knowledge base for LLMs working on Android code. Together, they provide comprehensive coverage of ADB, Gradle (build, test, lint via CLI), Kotlin (Compose skills), and Android SDK tools (emulator, avdmanager, sdkmanager). This is the single most important resource for OpenCode's Android support because it represents Google's official agent layer for Android development.

Skills repo: <https://github.com/android/skills> | **CLI docs:** <https://developer.android.com/tools/agents/android-cli>

Android Studio Gemini Agent Mode + MCP Support

Android Studio now includes built-in agentic AI in Gemini Agent Mode that can run builds, fix errors, generate tests, refactor code, and connect to external MCP servers. It supports AGENTS.md files for project-specific instructions and can add remote MCP servers without local installation. The native MCP support means that any MCP server (including the ADB and Gradle servers discussed below) can be connected directly within the IDE. For OpenCode, this is significant because it means that the MCP servers and skill definitions created for OpenCode's Android support will also work in Android Studio, creating a seamless developer experience across tools. The AGENTS.md pattern (project-level instruction files) should also be adopted by OpenCode as a project-level configuration complement to the global opencode.json.

Docs: <https://developer.android.com/studio/gemini/agent-mode>

5.2 Android MCP Servers

The MCP server ecosystem for Android development has exploded with multiple implementations covering different aspects of the Android development workflow. The following table provides a comprehensive comparison of the available Android MCP servers, their capabilities, and their suitability for OpenCode integration:

Project	Language	ADB	Gradle	Kotlin	UI Automation	Key Feature
mobile-next/mobile-mcp	Node.js	Yes	-	-	Yes	Platform-agnostic; emulators + real devices
IlyaGulya/gradle-mcp-server	Kotlin/JVM	-	Yes	-	-	Gradle Tooling API; list tasks, run builds
minhalvp/android-mcp-server	Python	Yes	-	-	Yes	UI dump via uiautomator; screenshots
github-hewei/mcp-android-adb	Go	Yes	-	-	-	Visual model integration for screen description
martingeidobler/android-mcp	Kotlin/Gradle	Yes	-	Yes	Yes	Most aligned with Android tech stack
CursorTouch/Android-MCP	Node.js	Yes	-	-	-	Lightweight; Cursor-optimized
Anjos2/mcp-android-emulator	Node.js	Yes	-	-	-	Redroid (containerized) emulator support

ulcica/android-mcp	Unknown	Yes	-	-	-	Deep device inspection and debugging
--------------------	---------	-----	---	---	---	--------------------------------------

Table 6. Android MCP Server Comparison

Recommended Integration Stack for Android MCP: Combine **IlyaGulya/gradle-mcp-server** for Gradle build interaction with **mobile-next/mobile-mcp** or **martingeidobler/android-mcp-server** for ADB device control. This gives a complete build-to-deploy pipeline: build (Gradle MCP) then deploy (ADB MCP). For Kotlin-specific agent workflows, **JetBrains Koog** (discussed below) provides a Kotlin-native agent framework.

5.3 Android Skill Definitions for AI Agents

amit-nayar/android-adb-skill

This project packages Android automation for AI agents in two layers: a shared execution surface at `./tools/android` (screenshots, UI inspection, gestures, app management, debugging) and agent-consumable skill definitions in the `skills/` directory. Critically, it works **without an MCP server** using a pure tool/skill approach. This makes it an excellent template for OpenCode because it demonstrates how to define ADB tools as agent-consumable skill definitions that can be directly referenced in the `opencode.json` configuration without requiring a separate MCP server process. The project provides tool definitions for device control, UI inspection, and app management that can be copied into OpenCode's `tools/` directory and referenced by the Android agent.

GitHub: <https://github.com/amit-nayar/android-adb-skill>

JetBrains Koog

Koog is JetBrains' open-source Kotlin Multiplatform framework for building AI agents with graph-based workflows. It includes MCP tool support, RAG, memory, and reusable strategies, and is battle-tested on JetBrains products. It supports JVM, Android, iOS, JS, and WASM targets. For OpenCode, Koog is significant because it provides a path to building Android-aware agents **in Kotlin** rather than Python or TypeScript, which aligns naturally with Android development teams. It can wrap ADB and Gradle commands as Koog tools and is MCP-compatible, meaning it can both serve and consume MCP tools. This could be the foundation for a Kotlin-native OpenCode agent SDK that Android developers can extend using the language they already know.

GitHub: <https://github.com/JetBrains/koog> | **Docs:** <https://docs.koog.ai>

6. Integration Recommendations & Roadmap

Based on the comprehensive analysis of 30+ open-source projects and architectural patterns, this section presents a prioritized, phased integration roadmap for OpenCode. Each phase is designed to be independently valuable while building toward the complete vision of a configurable, extensible, Android-aware AI agent platform.

6.1 Phase 1: Quick Wins (1-2 Weeks)

These are high-impact changes that require minimal architectural changes and can be implemented quickly by leveraging existing projects and patterns. They address the most pressing gaps with the least effort.

- **Add Android MCP servers to opencode.json** - Configure IlyaGulya/gradle-mcp-server and mobile-next/mobile-mcp as MCP servers in the default configuration. This immediately provides Gradle build and ADB device control capabilities without any code changes.
- **Adopt Google Android Skills** - Import SKILL.md files from android/skills into the OpenCode skills directory, providing official best-practice knowledge for Compose, Gradle, testing, and architecture.
- **Enhance the android-kotlin agent** - Replace the current 929-byte prompt definition with a structured agent definition using the CrewAI persona pattern (role, goal, backstory) plus explicit tool references to the newly added MCP servers.
- **Add schema versioning to config** - Following Continue's pattern, add a "schema: v2" field to opencode.json to enable backward-compatible config evolution.
- **Add AGENTS.md support** - Following Android Studio's pattern, support project-level AGENTS.md files that complement the global opencode.json with project-specific instructions.

6.2 Phase 2: Structural Improvements (2-4 Weeks)

These changes require more substantial refactoring but are essential for long-term extensibility and align OpenCode with emerging industry standards.

- **Migrate config to YAML** - Following Goose and Continue's example, migrate opencode.json to opencode.yaml while maintaining backward compatibility (auto-detect format). YAML provides comments, better readability for complex nested structures, and easier manual editing.
- **Formalize the SKILL.md standard** - Standardize the YAML frontmatter schema for SKILL.md files to include: name, description, version, toolkit_refs (referencing named toolkits), dependencies (other skills), and compatibility (supported agents). This enables skill versioning and dependency resolution.

- **Implement model roles and capabilities** - Following Continue's pattern, add declarative roles (chat, edit, plan, build) and capabilities (tool_use, image_input, streaming) arrays to model definitions. This enables automatic model selection based on task requirements.
- **Create Android Toolkit abstraction** - Define a named "android" toolkit that bundles ADB tools, Gradle MCP tools, and Kotlin compiler tools. Reference this toolkit from the android-kotlin agent's skill definitions.
- **Add custom provider files** - Following Goose's pattern, support self-describing provider JSON/YAML files in a providers/ directory that can be added without code changes.

6.3 Phase 3: Ecosystem Integration (4-8 Weeks)

These are larger-scale changes that transform OpenCode from a standalone tool into a platform with a thriving ecosystem of community-contributed skills, tools, and agent configurations.

- **Build a skill/tool registry** - Create a community registry (inspired by MCP Official Registry, Smithery, and AgentRegistry) where skills, toolkits, and MCP server configurations can be published, discovered, and installed. Start with a simple GitHub-based registry (like VoltAgent/awesome-agent-skills) and evolve toward a governed registry with validation.
- **Implement config auto-generation** - Following Caliber's pattern, build a project fingerprinting system that detects project type (Android, Laravel, React, etc.) and auto-generates appropriate agent configs, skill references, and MCP server configurations.
- **Protocol-agnostic tool registry** - Following ToolRegistry's pattern, implement a tool definition layer where tools are registered once with type annotations and automatically exported to OpenAI function calling, Anthropic tool use, and MCP formats.
- **Agent Card / persona standard** - Following Google A2A and CrewAI, implement structured Agent Card definitions that describe agent capabilities, skills, and authentication requirements. This enables inter-agent discovery and collaboration.
- **SDK for external integrations** - Expose a clean API for external tools to register agents, skills, and MCP servers programmatically, enabling third-party integrations without forking the codebase.

6.4 Recommended Android Integration Architecture

For the specific Android development workflow, the following architecture combines the best tools and patterns from the projects surveyed. This provides a complete build-test-deploy-verify pipeline for Android development within OpenCode:

Layer	Tool/Pattern	Source Project	Purpose
Knowl edge	Android Skills (SKILL.md)	android/skills	Official best-practice instructions for Compose, Gradle, testing

Build	Gradle MCP Server	IlyaGulya/gradle-mcp-server	Programmatic Gradle builds, task listing, dependency management
Deploy	Mobile MCP Server	mobile-next/mobile-mcp	ADB device control, app installation, emulator management
Inspect	ADB Skill Definitions	amit-nayar/android-adb-skill	Tool definitions for screenshots, UI dump, debugging
Agent	Android Agent (enhanced)	CrewAI persona pattern	Role/goal/backstory agent with Android toolkit refs
Config	Auto-generated opencode.yaml	Caliber pattern	Auto-detect Android project and configure tools
IDE	AGENTS.md + MCP servers	Android Studio pattern	Project-level instructions + MCP server references

Table 7. Recommended Android Integration Architecture for OpenCode

7. Complete Project Reference

This section provides a consolidated reference of all open-source projects analyzed in this report, organized by category with key details for quick lookup.

7.1 Agent Config & Registry Projects

Project	GitHub URL	Key Pattern	Relevance to OpenCode
Goose	github.com/aaif-goose/goose	MCP extensions + SKILL.md + YAML config	Highest: full-stack reference for config, skills, providers
Continue	github.com/continuedev/continue	YAML config + model roles + capabilities	High: config structure, schema versioning, prompt templates
Cline	github.com/cline/cline	MCP Marketplace + SDK	High: marketplace pattern for tool discovery
OpenHands	github.com/All-Hands-AI/OpenHands	Action/Observation typed loop	High: typed tool interface, ACI abstraction
CrewAI	github.com/crewaiinc/crewai	Persona-first agents + YAML crews	High: agent persona structure, per-agent tool assignment
Caliber	github.com/caliber-ai-org/ai-setup	Config auto-generation + sync	High: auto-config from project context

ToolRegistry	github.com/Oaklight/ToolRegistry	Protocol-agnostic tool registry	High: define once, export to multiple formats
Docker agent	github.com/docker/docker-agent	Pure YAML agent definitions	Medium: declarative agent + tool-as-command pattern
AgentDock	github.com/AgentDock/AgentDock	Framework-agnostic + configurable determinism	Medium: node-based composition, declarative agents
MCP Registry	github.com/modelcontextprotocol/registry	Official MCP server registry	High: discover and install MCP servers
Smithery	github.com/smithery-ai	MCP registry + installer	Medium: one-line MCP server installation
A2A Protocol	github.com/a2aproject/A2A	Agent Card + inter-agent communication	Medium: agent discovery and collaboration

Table 8. Agent Config and Registry Projects

7.2 Android-Aware Tooling Projects

Project	GitHub URL	Coverage	Integration Method
Android CLI + Skills	github.com/android/skills	ADB + Gradle + Kotlin + SDK	SKILL.md import + CLI tool definition
mobile-mcp	github.com/mobile-next/mobile-mcp	ADB + UI automation	MCP server in opencode.json
gradle-mcp-server	github.com/IlyaGulya/gradle-mcp-server	Gradle builds	MCP server in opencode.json
android-adb-skill	github.com/amit-nayar/android-adb-skill	ADB + UI inspection	Tool/skill copy into tools/directory
Koog	github.com/JetBrains/koog	Kotlin agent framework	Gradle dependency for custom agents
android-mcp-server	github.com/minhalvp/android-mcp-server	ADB + screenshots + UI dump	MCP server in opencode.json
mcp-android-adb	github.com/github-hewei/mcp-android-adb-server	ADB + visual model	MCP server in opencode.json
android-mcp (Kotlin)	github.com/martingeidobler/android-mcp-server	ADB + Kotlin + UI automation	MCP server in opencode.json

Android Studio Agent	developer.android.com/studio/gemini	Full IDE + MCP support	AGENTS.md + shared MCP servers
----------------------	-------------------------------------	------------------------	--------------------------------

Table 9. Android-Aware Tooling Projects

8. Key Takeaways

This research reveals several transformative insights for the OpenCode project. The AI agent ecosystem is converging on a set of standards and patterns that OpenCode can adopt to close its three critical gaps and establish itself as a leading configurable agent platform. The following takeaways represent the most important strategic insights from this analysis:

- MCP is the universal tool integration standard.** Nearly every major project (Cline, Goose, OpenAI Agents SDK, LangChain, AgentRegistry) now supports MCP. OpenCode already has MCP support in its config; the next step is to make MCP the primary extension mechanism by providing auto-discovery of MCP servers from the official registry and building a community-curated list of recommended MCP servers for common workflows.
- SKILL.md is the emerging cross-tool standard.** Adopted by Claude Code, Codex, Gemini CLI, Cursor, and Goose, the SKILL.md format (YAML frontmatter + Markdown body) is becoming the de facto standard for skill definitions. OpenCode already uses this format partially; formalizing it with a standard schema (including toolkit_refs, dependencies, and version) would make OpenCode skills portable across the entire ecosystem.
- YAML wins for human-editable config.** The migration from JSON to YAML across multiple projects (Continue, Goose, CrewAI, Docker cagent) is not coincidental. YAML supports comments, has better readability for nested structures, and is more forgiving for manual editing. OpenCode should migrate its main config from JSON to YAML while maintaining backward compatibility.
- Android tooling is ready for prime time.** Google's official Android CLI, Android Skills, and Android Studio Agent Mode with MCP support represent a turning point for Android-aware AI agents. Combined with the 6+ community MCP servers for ADB and the Gradle MCP Server, OpenCode can build a complete Android development pipeline by simply configuring existing MCP servers and importing existing skill definitions.
- Protocol-agnostic tool registries solve fragmentation.** ToolRegistry's approach of "register once, expose via multiple protocols" addresses the real problem of tool definitions being locked to specific LLM provider formats. As OpenCode supports multiple LLM providers, adopting this pattern would ensure that tool definitions work across all supported models without duplication.
- Config auto-generation lowers the barrier to entry.** Caliber's approach of fingerprinting projects and auto-generating agent configurations is a paradigm shift. Instead of requiring users to manually craft complex YAML/JSON configurations, OpenCode could detect project types and auto-suggest appropriate agents, tools, and skills. This would be especially impactful for Android

development, where the toolchain configuration is complex and error-prone.